AI Medical Report Simplifier — Team Role Assignments

6-person hackathon team. Each person picks ONE role below. Fields are labeled consistently (GOAL, TASKS, DELIVERABLE, TOOLS, EST. HOURS) so this document can be pasted into an AI tool to auto-generate a task spreadsheet.

ROLE 1: Local AI Setup

GOAL

Run a local model that reads X-ray images and medical reports and outputs clean structured text.

TASKS

1. Pick and install a local model (vision model for X-rays, OCR for scanned reports, e.g. Tesseract or a local VLM).
2. Set up the local inference environment (Python env, model weights, GPU/CPU config).
3. Write a script: input file (image/PDF) -> raw extracted text/findings.
4. Test on 5-10 sample reports and X-rays; fix garbled or missing output.
5. Define the exact output format (plain text or JSON) that Backend will consume.
6. Document setup steps so teammates can run it locally too.

DELIVERABLE	A working script/function: file in, structured text out. Shared with Backend owner.
TOOLS	Python, Tesseract OCR / local vision model (e.g. LLaVA, Ollama), OpenCV for image prep
EST. HOURS	6-8

ROLE 2: Backend / API Integration

GOAL

Connect the local model's output to the OpenAI API and return the simplified result to the frontend.

TASKS

1. Set up the OpenAI API key securely (environment variable, never hardcoded).
2. Build a small server/endpoint (Flask/FastAPI) that accepts the local model's output.
3. Send that text to OpenAI's API using the simplification prompt from Prompt Engineering.
4. Handle errors: API timeout, empty input, rate limits.
5. Return the simplified response in a clean JSON format to the frontend.
6. Log requests locally for debugging (no need for a database for a hackathon).

DELIVERABLE	A working API endpoint: local-model text in, simplified explanation out, ready for frontend to call.
TOOLS	Python (Flask or FastAPI), OpenAI API, requests library
EST. HOURS	6-8

ROLE 3: Prompt Engineering

GOAL
Write and refine the prompt that turns medical jargon into plain language an elderly patient can understand.

TASKS
1. Draft a system prompt: simple words, short sentences, no jargon left unexplained.
2. Add a rule: explain findings, never diagnose or give medical advice.
3. Add a closing line every time: 'Please discuss this with your doctor.'
4. Test the prompt against 5+ real or sample report texts from Data & Privacy.
5. Adjust tone: calm, factual, not alarming, not falsely reassuring.
6. Hand the final prompt string to Backend for integration.

DELIVERABLE	Final prompt text (ready to paste into the API call) plus 3-5 before/after test examples.
TOOLS	OpenAI Playground or ChatGPT for iteration, sample reports from Data & Privacy
EST. HOURS	5-6

ROLE 4: Frontend / UI

GOAL
Build a simple upload-and-result screen designed for an elderly user.

TASKS
1. Design one screen: upload button + result display (keep it to 1-2 screens total).
2. Use large fonts, high contrast, minimal buttons, no clutter.
3. Connect the upload action to Backend's API endpoint.
4. Display the simplified explanation clearly once it returns.
5. Add a loading state (processing can take a few seconds).
6. Optional if time allows: text-to-speech / 'read aloud' button.

DELIVERABLE	A working web page or app screen: upload a file, see the simplified result.
TOOLS	HTML/CSS/JS or React, fetch/axios to call the Backend API
EST. HOURS	6-8

ROLE 5: Data & Privacy

GOAL
Provide safe sample reports/X-rays for testing and make sure no real patient data is exposed.

TASKS
1. Source or create 5-10 sample X-ray images and medical report texts (synthetic or public/redacted).
2. Remove all identifying info (name, DOB, ID numbers) from any real-looking samples.
3. Write a short PII checklist the team follows before sending anything to OpenAI's API.
4. Test edge cases: blurry scan, handwriting, unusual report format.
5. Share the sample set with Local AI Setup and Prompt Engineering for testing.
6. Prepare one sentence for the pitch on how patient privacy is protected.

DELIVERABLE	A folder of test samples (privacy-safe) + a short privacy checklist doc.
TOOLS	Public/synthetic medical datasets, basic image/text editors
EST. HOURS	4-6

ROLE 6: Presentation / Pitch

GOAL

Prepare the demo and slides that explain the problem, solution, and live it in front of judges.

TASKS

1. Write the pitch narrative: problem (elderly patients don't understand reports) -> solution -> demo.
2. Build a short slide deck (problem, solution, how it works, tech stack, demo, impact).
3. Prepare a live or recorded demo run-through with the team.
4. Anticipate judge questions (privacy, accuracy, medical liability) and prep answers.
5. Float between teammates once the deck is done to help test and debug.
6. Time the pitch and rehearse at least twice before presenting.

DELIVERABLE	A ready-to-present slide deck plus a rehearsed demo script.
TOOLS	Google Slides / PowerPoint, notes app for Q&A; prep
EST. HOURS	5-7

How to use this with an AI tool

1. Pick the ROLE section that is yours.
2. Send this PDF (or just your role's section) to an AI chat tool.
3. Use a prompt like the one below to get a spreadsheet-ready task list.

SUGGESTED PROMPT

"Here is my role from our hackathon team-role PDF: [ROLE NAME]. Read the GOAL, TASKS, DELIVERABLE, TOOLS, and EST. HOURS fields for this role only. Turn the TASKS list into a spreadsheet with columns: Task, Subtasks, Tool/Resource, Estimated Hours, Status (Not started / In progress / Done), Notes. Split each task into 2-3 concrete subtasks. Keep it realistic for a hackathon timeline."